

Financial Advisor Portfolio Management System

Entity Relationship Diagram – Final Design Document

****Author:**** Entry-Level Software Developer, Investment Management Technology
****Organization:**** Wells Fargo
****Date:**** June 10, 2026
****Status:**** Final

1. Entity Identification

Four strong entities, each with its own UUID primary key:

Entity	Description
FINANCIAL_ADVISOR	Licensed advisor who manages client accounts
CLIENT	Individual investor whose assets are managed
PORTFOLIO	The single collection of securities owned by a client
SECURITY	An individual investment instrument held within a portfolio

2. Relationships

Relationship	Cardinality	Crow's Foot	Description
FINANCIAL_ADVISOR → CLIENT	One-to-Many	--o{	One advisor manages zero or more clients
CLIENT → PORTFOLIO	One-to-One	--	Each client owns exactly one portfolio
PORTFOLIO → SECURITY	One-to-Many	--o{	One portfolio contains zero or more securities

3. ERD – Mermaid.js (Visual Rendering)

```

erDiagram
    FINANCIAL_ADVISOR {
        uuid advisor_id PK
        varchar first_name
        varchar last_name
        varchar email
        timestamp created_at
        timestamp updated_at
    }
    CLIENT {
        uuid client_id PK
        uuid advisor_id FK
        varchar first_name
        varchar last_name
        varchar email
        timestamp created_at
        timestamp updated_at
    }
    PORTFOLIO {
        uuid portfolio_id PK
        uuid client_id FK
        varchar name
        timestamp created_at
        timestamp updated_at
  
```

```

    }

    SECURITY {
        uuid          security_id    PK
        uuid          portfolio_id   FK
        varchar       name
        varchar       category
        date          purchase_date
        decimal       purchase_price
        integer       quantity
        timestamp     created_at
        timestamp     updated_at
    }

    FINANCIAL_ADVISOR ||--o{ CLIENT      : "manages"
    CLIENT            ||--|| PORTFOLIO   : "owns"
    PORTFOLIO         ||--o{ SECURITY    : "contains"
...

```

> **Rendering tip:** Paste this block into mermaid.live, Miro, or any Markdown viewer with Mermaid support (VS Code, GitHub, Notion) to see the visual diagram.

4. ERD – PostgreSQL DDL (Relational Database)

```

```sql
-- =====
-- FINANCIAL_ADVISOR
-- Root entity. No foreign keys.
-- =====
CREATE TABLE financial_advisor (
 advisor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 first_name VARCHAR NOT NULL,
 last_name VARCHAR NOT NULL,
 email VARCHAR UNIQUE NOT NULL,
 created_at TIMESTAMP NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- =====
-- CLIENT
-- References: financial_advisor
-- =====
CREATE TABLE client (
 client_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 advisor_id UUID NOT NULL
 REFERENCES financial_advisor(advisor_id)
 ON DELETE CASCADE,
 first_name VARCHAR NOT NULL,
 last_name VARCHAR NOT NULL,
 email VARCHAR UNIQUE NOT NULL,
 created_at TIMESTAMP NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- FK index: supports JOIN on advisor_id and advisor workload queries
CREATE INDEX idx_client_advisor_id ON client(advisor_id);

-- =====
-- PORTFOLIO
-- References: client

```

-- 1:1 enforced by UNIQUE constraint on client\_id

```
-- =====
CREATE TABLE portfolio (
 portfolio_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 client_id UUID UNIQUE NOT NULL
 REFERENCES client(client_id)
 ON DELETE CASCADE,
 name VARCHAR NOT NULL,
 created_at TIMESTAMP NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

-- Note: The UNIQUE constraint on client\_id automatically creates a  
 -- unique index in PostgreSQL. No separate CREATE INDEX is needed.

-- =====  
 -- SECURITY

-- References: portfolio

```
-- =====
CREATE TABLE security (
 security_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 portfolio_id UUID NOT NULL
 REFERENCES portfolio(portfolio_id)
 ON DELETE CASCADE,
 name VARCHAR NOT NULL,
 category VARCHAR NOT NULL, -- e.g. stock, bond, ETF, mutual fund
 purchase_date DATE NOT NULL,
 purchase_price DECIMAL(15,2) NOT NULL,
 quantity INTEGER NOT NULL,
 created_at TIMESTAMP NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

-- FK index: supports JOIN on portfolio\_id and holdings queries  
 CREATE INDEX idx\_security\_portfolio\_id ON security(portfolio\_id);  
 ````

5. Design Rationale

5.1 Why PORTFOLIO is a Strong Entity

A strong entity has a surrogate primary key and meaningful attributes that exist independently of the relationship that created it. `portfolio\_id` identifies the record, `name` describes it, and `created\_at`/`updated\_at` track its lifecycle – none of these depend on `CLIENT` for their meaning. A weak entity would instead use a composite key such as `(client\_id, portfolio\_seq)` and could not be cleanly referenced by `SECURITY` without dragging `CLIENT` into every join. Modeling `PORTFOLIO` as strong keeps foreign key references clean, supports future extension (rebalancing history, audit logs, benchmark assignments), and maintains schema symmetry across the four entities.

5.2 Why CLIENT–PORTFOLIO is 1:1 and Not 1:N

The requirement specifies a single unified view of each client's holdings. Permitting multiple portfolios per client would introduce unanswered questions – which portfolio is primary, how to aggregate total exposure, how to render a default view – without any stated business justification for that complexity. The `UNIQUE NOT NULL` constraint on `portfolio.client\_id` enforces this cardinality at the database level, making it structurally impossible for application code to accidentally create a second portfolio for the same client.

5.3 Intentionally Excluded Concerns

Three categories were left out of scope:

| Category | Examples | Reason Excluded |
|-----------------------------------|--|--|
| ***Infrastructure & application** | Authentication, sessions, RBAC, notification queues | Runtime concerns belonging to application and infrastructure layers, not the data model |
| ***Derived & volatile data** | Current market price, unrealized gain/loss, total return | Market price changes continuously and is fetched from a live feed at runtime; performance metrics are computed from `purchase_price`, `quantity`, and live prices – not stored |
| ***Historical state** | Advisor–client assignment history, portfolio ownership transfers | The model captures current state only; a compliance–grade audit log is a future iteration requiring its own specification |

6. Quality Checklist

| Check | Status |
|---|--------|
| Every entity has a UUID primary key | PASS |
| Every FK column has a corresponding index | PASS |
| No many-to-many relationships | PASS |
| SECURITY attributes match spec exactly: `name`, `category`, `purchase_date`, `purchase_price`, `quantity` | PASS |
| No infrastructure concerns in the data model | PASS |
| Crow's Foot cardinalities consistent between Mermaid and DDL | PASS |

7. ERD Notation Reference (Crow's Foot)

| Notation | Meaning |
|----------|-----------------------------|
| `  --  ` | Exactly one to exactly one |
| `  --o{` | Exactly one to zero-or-many |
| `  -- {` | Exactly one to one-or-many |
| `o --o{` | Zero-or-one to zero-or-many |

Document generated for the Wells Fargo Investment Management Technology team.
 Data model only – infrastructure, UI, and application architecture are addressed separately.